

PROTOCOL SIMULATORS

Project Documentation: Tools & Technologies and Application Creation Prompts

Author:	Jonak Pratim Thakuria	Roll Number:	202402021060
Repository:	https://github.com/Xingho1/ProtocolSim	Live Production URL:	https://protocolsim.onrender.com/

1. PROJECT & AUTHOR INFORMATION

- **Author:** Jonak Pratim Thakuria
- **Roll Number:** 202402021060
- **Repository:** <https://github.com/Xingho1/ProtocolSim>
- **Live Production URL:** <https://protocolsim.onrender.com/>

2. AI PLATFORMS USED

- **ChatGPT (OpenAI):** Architecture design, concept ideation, and initial prototyping.
- **Antigravity (Google DeepMind):** Full-stack agentic development, debugging, iterative refactoring, UI overhaul, deployment readiness, and protocol packet simulation.

3. TECHNOLOGIES & STACK

3.1 Backend Core & API Architecture

- **Python 3.10+:** Core programming language powering both backend network engines and frontend visualization.
- **FastAPI:** Modern, high-performance web framework for building REST APIs and asynchronous endpoints.
- **Uvicorn:** Lightning-fast ASGI web server implementation used for hosting FastAPI services.
- **Pydantic (v2):** Data validation, settings management, and structured request/response serialization.
- **Asyncio:** Python standard library for cooperative multitasking, asynchronous network operations, and concurrent requests.

3.2 Networking & Protocol Simulation Engine

- **Scapy:** Comprehensive packet manipulation, protocol header dissection, and packet layer analysis (DNS, TCP, HTTP, UDP, ICMP).
- **dnspython (dns.resolver, dns.message, dns.query):** Complete DNS protocol inspection, wire-format query generation, RRsets, TTL parsing, and response code validation (NOERROR, NXDOMAIN, etc.).
- **Socket Programming (socket):** Low-level UDP and TCP socket manipulation for simulating chunk-based, real-time packet transmission and latency emulation.
- **smtplib & ssl:** Standard SMTP protocol client simulating email transactions with STARTTLS encryption negotiation and MIME message handling.

- **Requests / HTTPX:** HTTP client libraries used for HTTP/1.1 and HTTP/2 request lifecycle tracing, header exchanges, status code validation, and content retrieval.

3.3 Media Processing & Video Streaming

- **OpenCV (cv2):** Computer vision library for extracting video metadata (FPS, frame count, resolution), frame-by-frame decoding, JPEG compression, and chunked byte generation.
- **Custom UDP Video Streamer:** Protocol simulation engine breaking video assets into timestamped, sequence-numbered UDP packet payloads to demonstrate real-time packet loss, throughput, and jitter.

3.4 Frontend & User Interface

- **Streamlit:** Modern Python UI framework delivering interactive, reactive web dashboards.
- **Custom Modern CSS / Styling:** Handcrafted minimalistic light-mode theme featuring high-contrast clean typography, elevated status pill badges (SUCCESS, PENDING, FAILED), modular protocol pipeline cards, and a responsive dual-pane layout.
- **HTML5 & CSS Components:** Embedded custom renderers for HTML source previews and secure sandboxed iframes.

3.5 DevOps, Containerization & Deployment

- **Docker:** Containerization platform providing deterministic builds across all host operating systems.
- **Docker Compose:** Multi-container service definition and orchestration (backend FastAPI service + frontend Streamlit dashboard).
- **Procfile:** Process manager configuration for containerless cloud platforms (Heroku, Render, Railway).
- **Git & GitHub:** Distributed version control, collaborative branch management, and repository hosting.

4. PROJECT ARCHITECTURE OVERVIEW

```

Assignment/
├── backend/
│   ├── app.py                # FastAPI REST API & simulation endpoints
│   ├── protocol_engine/      # Modularized networking & protocol simulation
│   │   ├── browsing.py       # DNS resolution + TCP Handshake + HTTP/HTTPS fetch
│   │   ├── mail.py           # SMTP connection, STARTTLS handshake, MIME dispatch
│   │   ├── streaming.py      # UDP chunking, video frame pipeline, packet tracer
│   │   └── utils.py          # Packet dissection, header extraction, timing utilities
│   └── videos/               # Local video repository for UDP streaming testbench
├── frontend/
│   ├── app.py                # Main Streamlit dashboard UI
│   ├── components.py         # Visual UI widgets, status cards, protocol inspection cards
│   └── styles.py              # Custom light-mode CSS design system
├── run.py                    # Single-command unified dual-process orchestrator
├── Dockerfile                 # Production multi-stage Docker build specification
├── docker-compose.yml         # Orchestration file for local & cloud containers
├── Procfile                   # Heroku / Render deployment process file
├── requirements.txt           # Pinned Python package dependencies
└── README.md                  # Setup and execution guide

```

5. KEY FEATURES IMPLEMENTED

1. Browsing Simulation (DNS + TCP + HTTP):

- Real-world DNS lookups with complete header and RRset decomposition.
- 3-Way TCP Handshake simulation (SYN, SYN-ACK, ACK).
- HTTP/HTTPS request and response negotiation with source rendering.

2. Email Simulation (SMTP + TLS):

- End-to-end SMTP mail transaction tracing (EHLO, STARTTLS, AUTH, MAIL FROM, RCPT TO, DATA, QUIT).
- Mock mode for offline demonstrations and live relay mode for real SMTP server testing.

3. UDP Video Streaming Simulation:

- Server-side video chunking and frame packetization.
- Simulation of realistic packet flow, sequence numbers, jitter, and packet delivery.

4. Dual-Pane Interactive Pipeline:

- Left panel: Configuration, input parameters, and protocol controls.
- Right panel: Visual, simplified step-by-step protocol flow with clear success/failure indicators.

6. COMPLETE APPLICATION CREATION PROMPTS

Total Prompts Recorded: 30

PROMPT #1

Create a dual-panel web dashboard in Python (FastAPI + Streamlit).

Left panel: Activity selector with three modes -

- Browsing (URL input + Visit button)
- Mail (compose form: To, Subject, Body + Send button)
- Streaming (Play / Pause + quality selector)

Right panel: Live protocol visualizer that updates in parallel with the left panel:

- Browsing → show DNS query/response followed by full HTTP request/response
- Mail → show complete SMTP conversation (EHLO, MAIL FROM, RCPT TO, DATA, QUIT)
- Streaming → show DNS + HTTP playlist/manifest request + segment requests

Make the right panel animated or step-by-step (pause / next / previous / replay). Keep both panels synchronized so that every user action on the left immediately triggers the corresponding protocol visualization on the right.

PROMPT #2

down the server

PROMPT #3

I want to use terminal to run my app

PROMPT #4

Now, can we do these?

- Browser, make an HTML renderer or simple browser viewer for the fetched data,
- Mail, make that mail send to the To email,
- Streaming, will playing the video work for YouTube?

Note: Corrected 'viewr' to 'viewer', and 'will play the video works for youtube?' to 'will playing the video work for YouTube?'

PROMPT #5

@[backend/protocol_engine.py] divide this code here into modules for each action

Note: Corrected 'module of each action' to 'modules for each action'

PROMPT #6

@[f:\Assignment\backend\protocol_engine\utils.py:L7-L16] what is it doing?

PROMPT #7

@[f:\Assignment\backend\protocol_engine\browsing.py:L17]

If DNS request fails or the address actually doesn't exist then do show failed and respective error

PROMPT #8

@[backend/protocol_engine/mail.py]

make it simplified, if toggle for real mail service is on then, if the mail is not sent raise error, and if it's turned off, then simulate mailing is done, note that commands used should be shown too

Note: Corrected 'toggole' to 'toggle', 'an if its' to 'and if it's', 'commans uses' to 'commands used', 'should be show too' to 'should be shown too'

PROMPT #9

@[backend/protocol_engine/streaming.py] and @[backend/protocol_engine/youtube.py]

can we not do real streaming of YouTube video while using UDP etc?

Note: Corrected 'youtube' to 'YouTube'

PROMPT #10

if I have a video in the server part of the code, can we stream it using UDP?

Note: Corrected 'i' to 'I', 'streaming' to 'stream', and 'uDP' to 'UDP'

PROMPT #11

okay build it, make a system such that, there is a folder where I can put the videos to be streamed and then it is streamed to my frontend, no need for simulator and YouTube

Note: Corrected 'i' to 'I', 'and the it is' to 'and then it is', and 'youtube' to 'YouTube'

PROMPT #12

get rid of the YouTube streaming

Note: Corrected 'riid' to 'rid', and 'youtube' to 'YouTube'

PROMPT #13

why do I not feel that packet is transmitting?

also get rid of any unnecessary code from HLS streaming

Note: Corrected 'unnecary' to 'unnecessary', and 'HSL' to 'HLS'

PROMPT #14

make the streaming such that,

as the video is sent to the front end I can control the speed, when video is playing every 1 sec 2 packets are sent in real time

Note: Corrected 'i' to 'I', 'when video is play' to 'when video is playing', and 'Isec 2packaets' to '1 sec 2 packets'

PROMPT #15

revoke all changes just done

PROMPT #16

now in the front end,
make the design minimalistic, with only light version.
Make a very simple header, Protocol Simulators.
add borders to both the panes

Note: Corrected 'esign' to 'design', and 'borders two both' to 'borders to both'

PROMPT #17

now for the left pane,
Make the protocol selection part more highlighted,
then,
1. Browsing tab,
make the HTML Source view scrollable,
2. Mail
highlight the headings a little
3. Streaming
get rid of the rescan
give a tooltip for Active range and chunk
fix the poorly working stream chunk controls (check if they are actually working or part of simulation.) if they are part of the simulation add details about how it works and what it means.

other than these changes, add a loading animation when it is loading

Note: Corrected 'HTML Scorce' to 'HTML Source', 'tool tip' to 'tooltip', and 'woprking' to 'working'

PROMPT #18

put the videos dir inside the backend

Note: Corrected 'backen' to 'backend'

PROMPT #19

@[f:\Assignment\frontend\app.py:L325] how is this function working, shouldn't there be a fetch for the video through the protocol like actual streaming?

PROMPT #20

do it

PROMPT #21

update whatever you like make the streaming realistic live video streaming. if the packets do not arrive video will not play. I can stop the inflow of packets as I wish with a button

Note: Corrected 'realastics' to 'realistic', 'viedo' to 'video', and 'as i ish' to 'as I wish'

PROMPT #22

revoke last change

PROMPT #23

get rid of Stream Chunk Controls and related

PROMPT #24

@[f:\Assignment\frontend\app.py:L386-L395] what does this do?
also get rid of Active Range, Size from video details

PROMPT #25

okay, now in the right pane simply show the steps of the protocol pipeline, in simple chunks, it should be very simplified

Note: Corrected 'shoul' to 'should', 'ery' to 'very', and 'simpilified' to 'simplified'

PROMPT #26

show card for each step the @[f:\Assignment\frontend\app.py:L434-L457] this part doesn't seem to work

Note: Corrected 'does't' to 'doesn't'

PROMPT #27

FAILED (NOERROR) why am I getting this error in
Step 02

DNS

Server → Client

FAILED (NOERROR)

PENDING

Standard query response 0x5aba A example.com -> 172.66.147.243

DNS resolver returns A record resolving 'example.com' to 172.66.147.243 (TTL: 300s)

Note: Corrected 'why m i getting' to 'why am I getting'

PROMPT #28

now optimise the code for deployment

Note: Corrected 'coe' to 'code'

PROMPT #29

now where and how should I deploy?

Note: Corrected 'i' to 'I'

PROMPT #30

now in "F:\" make a txt file called prompts.txt where all the prompts are stored from this application creation.

Note: Corrected 'calle' to 'called'